

# Data Analysis with Python 2024–2025

## Parte 1: Python Conteúdo Teórico

Tradução e Adaptação: University of Helsinki — MOOC.fi

### Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

#### O que você vai aprender nesta página

- Aprender os conceitos básicos da linguagem Python.
- Compreender strings e módulos.
- Trabalhar com sequências e objetos iteráveis.

### Conteúdo

## 1 Entrada e saída básicas

O programa clássico “Hello, world!” é extremamente simples em Python. A função `print()` envia texto para a saída padrão. Vários argumentos podem ser passados para `print`; por padrão, eles são separados por espaços. Expressões numéricas são avaliadas antes de serem impressas.

```
print("Hello world2!")
print("Hello,", "John!", "How are you?")
print(1, "plus", 2, "equals", 1+2)
```

A função `input()` permite ler texto digitado pelo usuário. O resultado de `input()` é uma string.

```
name = input("Give me your name: ")
print("Hello,", name)
```

### 1.1 Indentação

Python usa indentação para marcar blocos de código. Quatro espaços são uma convenção comum. Diferentemente de linguagens que usam chaves, o fim da indentação marca o fim do bloco.

```
for i in range(3):
    print("Hello")
print("Bye!")
```

A expressão `range(n)` produz os inteiros de 0 a  $n - 1$ ; o limite final não é incluído.

## 2 Variáveis e tipos de dados

Uma atribuição associa um nome a um valor. Python não exige a declaração prévia de uma variável nem a especificação do tipo. O tipo pertence ao objeto referenciado, e isso caracteriza a tipagem dinâmica.

Os tipos básicos destacados neste material incluem `int`, `float`, `complex`, `str`, `bool` e `bytes`. A função `type()` permite descobrir o tipo do valor atual de um nome.

```
a = 1
print(type(a))
a = "algum texto"
print(type(a))
```

Os construtores de tipo também podem ser usados para conversão, por exemplo `int()`, `float()`, `str()` e `bool()`.

Bytes representam unidades de informação de 8 bits. Em situações que envolvem arquivos ou conjuntos de dados, a codificação de texto pode precisar ser especificada. O exemplo de UTF-8 mostra que um caractere pode ocupar mais de um byte.

## 3 Strings

Strings são sequências de caracteres delimitadas por aspas simples ou duplas. Aspas podem ser escapadas com barra invertida, e sequências como `\n` e `\t` representam nova linha e tabulação.

Strings também podem ser escritas em múltiplas linhas usando aspas triplas. Embora seja possível concatenar com `+`, para muitas strings o método `join()` é preferível por eficiência.

### 3.1 Interpolação de strings

O material apresenta três formas: formatação tradicional, o método `format()` e f-strings. Atualmente, f-strings e `format()` são as formas mais usadas nos exemplos.

```
print("{} plus {} is equal to {}".format(1, 3, 4))
print(f"{1} plus {3} is equal to {4}")
print(f"{1.6:.1f} {1.7:.2f} {1.8:.3f}")
```

Especificadores permitem controlar casas decimais e largura de campos. Para strings, o especificador `s` pode ser usado; para inteiros, `d`; e para números de ponto flutuante, `f`.

## 4 Expressões e instruções

Uma expressão é um trecho de código que produz um valor. Exemplos incluem operações aritméticas, chamadas de funções, indexação, comparações e acesso a atributos.

Uma instrução é um comando que produz um efeito. Atribuições e chamadas de função isoladas são exemplos. Python possui operadores de atribuição aumentada como `+=`, `-=`, `*=`, `/=` e outros.

## 5 Laços para tarefas repetitivas

Python possui principalmente os laços `while` e `for`. Um `while` repete enquanto uma condição `for` verdadeira.

```
i = 1
while i*i < 1000:
    print("Square of", i, "is", i*i)
    i += 1
```

O `for` percorre os elementos de um iterável. Listas e objetos criados por `range()` são exemplos.

```
s = 0
for i in [0,1,2,3,4,5,6,7,8,9]:
    s += i
print("The sum is", s)
```

## 6 Decisões com if

A estrutura `if/elif/else` seleciona blocos de acordo com condições booleanas.

```
x = int(input("Give an integer: "))
if x >= 0:
    a = x
else:
    a = -x
print("The absolute value of %i is %i" % (x, a))
```

### 6.1 break e continue

`break` encerra o laço atual quando a condição desejada é encontrada. `continue` interrompe apenas a iteração corrente e passa para a próxima.

## 7 Funções

Funções são declaradas com `def`. Uma função pode possuir parâmetros, retornar um valor e conter uma docstring descrevendo sua finalidade.

```
def double(x):  
    # Multiplica o argumento por dois.  
    return x*2  
  
print(double(4))
```

Python permite parâmetros posicionais, parâmetros nomeados, valores padrão e empacotamento/desempacotamento com `*` e `**`. Funções criam um novo escopo local.

### 7.1 Escopo

Variáveis definidas dentro de uma função são locais. Variáveis globais são acessíveis de dentro da função, mas uma atribuição normalmente cria um nome local. A instrução `global` pode ser usada quando é realmente necessário reatribuir um global; em funções aninhadas, `nonlocal` referencia o escopo externo mais próximo.

## 8 Estruturas de dados

As principais estruturas destacadas são strings, listas, tuplas, dicionários e conjuntos.

### 8.1 Sequências

Listas podem ter quantidade arbitrária de elementos e são mutáveis. Tuplas são sequências ordenadas e imutáveis. Strings, listas e tuplas compartilham operações como `len()`, indexação, fatiamento, concatenação e repetição.

A indexação começa em zero. Índices negativos contam a partir do final. Slices seguem a forma `sequencia[inicio:fim:passo]`, com o limite final excluído.

### 8.2 Modificação de listas

Listas podem ser alteradas por indexação, slicing e métodos como `append`, `extend`, `insert`, `remove`, `pop`, `reverse` e `sort`. A função `sorted()` devolve uma nova lista ordenada sem modificar a original.

### 8.3 range e ordenação

`range()` produz uma sequência numérica eficiente sem materializar obrigatoriamente uma lista. Seu passo pode ser especificado. Para ordenar, `sort()` altera a lista e `sorted()` produz outra.

### 8.4 zip

`zip()` combina várias sequências. Se duas sequências forem combinadas, os elementos resultam em pares; com  $n$  sequências, o resultado tem tuplas de  $n$  elementos. O tamanho é limitado pela sequência mais curta.

### 8.5 enumerate

`enumerate()` fornece simultaneamente índice e elemento. Conceitualmente, é equivalente a combinar `range(len(L))` e `L` com `zip()`.

## 8.6 Dicionários

Dicionários associam chaves a valores. As chaves precisam ser hashable. A consulta é feita por indexação. Entre os métodos importantes estão `items()`, `keys()`, `values()`, `get()`, `update()`, `pop()` e `popitem()`.

## 8.7 Conjuntos

Conjuntos armazenam elementos únicos e suportam união, interseção, diferença e diferença simétrica. Os operadores correspondentes são `|`, `&`, `-` e `^`.

## 8.8 Operador in, desempacotamento e del

O operador `in` testa pertinência em contêineres. Sequências e dicionários também podem ser desempacotados em múltiplos nomes. A instrução `del` remove uma associação ou um item/slice de um contêiner.

# 9 Comprehensions

Comprehensions permitem construir estruturas de dados de maneira compacta. O material apresenta list comprehensions, dictionary comprehensions e set comprehensions, incluindo múltiplos laços e filtros.

# 10 Processamento de sequências

Funções podem ser tratadas como objetos de primeira classe: podem ser passadas como argumentos, retornadas por funções e armazenadas em variáveis ou estruturas de dados. Nesta parte são estudadas `map`, `filter`, `reduce` e expressões `lambda`.

## 10.1 map e lambda

`map()` aplica uma função a cada elemento e devolve um iterável. Em Python moderno, é comum convertê-lo explicitamente em lista quando se deseja visualizar ou armazenar os resultados.

```
def double(x):  
    return 2*x  
L = [12, 4, -1]  
print(list(map(double, L)))
```

Uma `lambda` define uma pequena função sem nome.

```
L = [2, 3, 5]  
print(list(map(lambda x: 2*x+x**2, L)))
```

## 10.2 filter

`filter()` conserva apenas os elementos para os quais a função predicado retorna `True`. O material observa que comprehensions são frequentemente mais flexíveis em Python moderno.

## 10.3 reduce

`reduce()` combina iterativamente elementos de um iterável até obter um único resultado. Em Python, está em `functools`. Pode receber um valor inicial.

## 11 Manipulação de strings

Strings são imutáveis. Seus métodos não alteram a string original; em vez disso, retornam novos valores. O material organiza os métodos em grupos: classificação, transformação, busca, ajuste e junção/divisão.

### 11.1 Classificação

Métodos como `isalpha()`, `isdigit()`, `isalnum()`, `islower()`, `isupper()` e `isspace()` produzem valores booleanos.

### 11.2 Transformação

`lower()`, `upper()`, `capitalize()`, `title()` e `swapcase()` são exemplos de transformação de texto.

### 11.3 Busca

Métodos como `find()`, `index()`, `count()`, `startswith()` e `endswith()` ajudam a localizar substrings.

### 11.4 Ajuste

`strip()`, `lstrip()` e `rstrip()` removem caracteres das extremidades. O argumento pode especificar um conjunto de caracteres.

### 11.5 join e split

`join()` combina uma sequência de strings usando um separador. Para muitas strings, ele é mais eficiente que repetidas concatenações com `+`. `split()` divide uma string em partes, por um separador ou por espaços em branco quando nenhum separador é fornecido.

## 12 Módulos

Módulos dividem programas grandes em unidades menores e independentes. Cada arquivo `.py` pode funcionar como um módulo. A biblioteca padrão possui centenas de módulos; entre os exemplos citados estão `re`, `math`, `random`, `os` e `sys`.

### 12.1 Usando módulos

A forma `import math` permite acessar funções com o nome do módulo, como `math.cos(0)`. Também é possível importar nomes específicos com `from math import cos`. Embora `from math import *` exista, essa forma pode causar confusão por importar muitos nomes para o escopo atual.

### 12.2 Localização de módulos

Ao importar um módulo, Python procura primeiro módulos embutidos e depois os caminhos de `sys.path`. Isso explica por que arquivos locais podem ser importados e por que colisões de nomes podem ocorrer.

### 12.3 Hierarquia de módulos e pacotes

Pacotes são módulos que podem conter outros módulos e subpacotes. A hierarquia do sistema de arquivos representa a hierarquia de importação. Historicamente, arquivos `__init__.py` são usados para compor um pacote.

### 12.4 Conteúdo de um módulo

Atribuições, classes e funções definidas em um arquivo tornam-se atributos do módulo. O atributo `__name__` vale `"__main__"` quando o arquivo é executado diretamente. Isso permite usar o padrão:

```
def main():
    for _ in range(3):
        print("Hello")

if __name__ == "__main__":
    main()
```

## 13 Resumo

Os blocos de código em Python são definidos por indentação consistente. O laço `for` percorre os elementos de um iterável sem exigir o controle explícito dos índices. Python possui tipagem dinâmica, bom suporte para manipulação de strings e uma grande variedade de ferramentas para processar sequências, como `map`, `filter`, `reduce`, `zip`, `enumerate` e `range`. O material também destaca o uso de `join()` para concatenar muitas strings e o papel das funções anônimas `lambda`.